

```
# =====
# MACHINE LEARNING PRACTICAL - MODULE II
# House Price Prediction using Regression Techniques
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures

from sklearn.linear_model import (
    LinearRegression,
    Ridge,
    Lasso
)

from sklearn.svm import SVR

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# -----
# STUDENT DETAILS
# -----

print("=" * 75)
print("MACHINE LEARNING PRACTICAL - MODULE II")
print("House Price Prediction using Regression Techniques")
print("=" * 75)

print("\nStudent Name : Pragati Parmar")
print("Roll Number : BETN1AI25100")
print("Program : B.Tech CSE (AI & ML)")
print("Subject : Machine Learning")
print("Dataset : Housing Dataset")

# -----
# LOAD DATASET
# -----

print("\n" + "=" * 75)
print("1. DATASET LOADING AND INSPECTION")
print("=" * 75)

HOUSING_URL = (
```

```
"https://raw.githubusercontent.com/"
"Pragati200706/Machine-Learning-Practical/"
"main/Housing.csv"
)

house = pd.read_csv(HOUSING_URL)

print("\nDataset Shape:")
print(house.shape)

print("\nFirst 10 Records:")
display(house.head(10))

print("\nLast 10 Records:")
display(house.tail(10))

print("\nDataset Information:")
house.info()

print("\nDescriptive Statistics:")
display(house.describe(include="all").T)

print("\nMissing Values:")
display(house.isnull().sum())

# -----
# DATA PREPARATION
# -----

print("\n" + "=" * 75)
print("2. DATA PREPARATION")
print("=" * 75)

# Separate input features and target variable

X = house.drop(
    "price",
    axis=1
)

y = house["price"]

print("\nTarget Variable:")
print("price")

print("\nOriginal Feature Shape:")
print(X.shape)

# -----
# CATEGORICAL ENCODING
# -----
```

```
categorical_columns = X.select_dtypes(
    include="object"
).columns.tolist()

numerical_columns = X.select_dtypes(
    exclude="object"
).columns.tolist()

print("\nCategorical Columns:")
print(categorical_columns)

print("\Numerical Columns:")
print(numerical_columns)

X = pd.get_dummies(
    X,
    drop_first=True
)

# Convert boolean values to numerical values
X = X.astype(float)

print("\nFeature Shape After Encoding:")
print(X.shape)

# -----
# FEATURE SCALING
# -----

scaler = StandardScaler()

X_scaled = pd.DataFrame(
    scaler.fit_transform(X),
    columns=X.columns
)

print("\nScaled Feature Sample:")
display(X_scaled.head())

# -----
# TRAIN TEST SPLIT
# -----

print("\n" + "=" * 75)
print("3. TRAIN-TEST SPLIT")
print("=" * 75)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.30,
    random_state=42
```

```
)

print("\nTraining Set:")
print(X_train.shape)

print("\nTesting Set:")
print(X_test.shape)

print("\nTraining Records:", len(X_train))
print("Testing Records :", len(X_test))

# -----
# HELPER FUNCTION FOR METRICS
# -----

def calculate_metrics(actual, predicted):

    mae = mean_absolute_error(
        actual,
        predicted
    )

    mse = mean_squared_error(
        actual,
        predicted
    )

    rmse = np.sqrt(mse)

    r2 = r2_score(
        actual,
        predicted
    )

    return mae, mse, rmse, r2

# =====
# MODEL 1 – SIMPLE LINEAR REGRESSION
# =====

print("\n" + "=" * 75)
print("4. SIMPLE LINEAR REGRESSION")
print("=" * 75)

simple_model = LinearRegression()

simple_model.fit(
    X_train[["area"]],
    y_train
)

simple_pred = simple_model.predict(
```

```
    X_test[["area"]]
)

simple_mae, simple_mse, simple_rmse, simple_r2 = calculate_metrics(
    y_test,
    simple_pred
)

print("\nModel: Simple Linear Regression")
print("MAE  :", simple_mae)
print("MSE  :", simple_mse)
print("RMSE :", simple_rmse)
print("R²   :", simple_r2)

# =====
# MODEL 2 – MULTIPLE LINEAR REGRESSION
# =====

print("\n" + "=" * 75)
print("5. MULTIPLE LINEAR REGRESSION")
print("=" * 75)

multiple_model = LinearRegression()

multiple_model.fit(
    X_train,
    y_train
)

multiple_pred = multiple_model.predict(
    X_test
)

multiple_mae, multiple_mse, multiple_rmse, multiple_r2 = calculate_metrics(
    y_test,
    multiple_pred
)

print("\nModel: Multiple Linear Regression")
print("MAE  :", multiple_mae)
print("MSE  :", multiple_mse)
print("RMSE :", multiple_rmse)
print("R²   :", multiple_r2)

# =====
# MODEL 3 – POLYNOMIAL REGRESSION
# =====

print("\n" + "=" * 75)
print("6. POLYNOMIAL REGRESSION")
print("=" * 75)
```

```
poly = PolynomialFeatures(
    degree=2,
    include_bias=False
)

X_train_poly = poly.fit_transform(
    X_train
)

X_test_poly = poly.transform(
    X_test
)

poly_model = LinearRegression()

poly_model.fit(
    X_train_poly,
    y_train
)

poly_pred = poly_model.predict(
    X_test_poly
)

poly_mae, poly_mse, poly_rmse, poly_r2 = calculate_metrics(
    y_test,
    poly_pred
)

print("\nModel: Polynomial Regression")
print("Degree:", 2)
print("MAE :", poly_mae)
print("MSE :", poly_mse)
print("RMSE :", poly_rmse)
print("R² :", poly_r2)

# =====
# MODEL 4 – RIDGE REGRESSION
# =====

print("\n" + "=" * 75)
print("7. RIDGE REGRESSION")
print("=" * 75)

ridge_model = Ridge(
    alpha=1.0
)

ridge_model.fit(
    X_train,
    y_train
)
```

```
ridge_pred = ridge_model.predict(
    X_test
)

ridge_mae, ridge_mse, ridge_rmse, ridge_r2 = calculate_metrics(
    y_test,
    ridge_pred
)

print("\nModel: Ridge Regression")
print("Alpha:", 1.0)
print("MAE :", ridge_mae)
print("MSE :", ridge_mse)
print("RMSE :", ridge_rmse)
print("R² :", ridge_r2)

# =====
# MODEL 5 - LASSO REGRESSION
# =====

print("\n" + "=" * 75)
print("8. LASSO REGRESSION")
print("=" * 75)

lasso_model = Lasso(
    alpha=1000,
    max_iter=20000
)

lasso_model.fit(
    X_train,
    y_train
)

lasso_pred = lasso_model.predict(
    X_test
)

lasso_mae, lasso_mse, lasso_rmse, lasso_r2 = calculate_metrics(
    y_test,
    lasso_pred
)

print("\nModel: Lasso Regression")
print("Alpha:", 1000)
print("MAE :", lasso_mae)
print("MSE :", lasso_mse)
print("RMSE :", lasso_rmse)
print("R² :", lasso_r2)

# =====
# MODEL 6 - SUPPORT VECTOR REGRESSION
```

```
# =====

print("\n" + "=" * 75)
print("9. SUPPORT VECTOR REGRESSION")
print("=" * 75)

# Scale target variable for SVR

target_scaler = StandardScaler()

y_train_scaled = target_scaler.fit_transform(
    y_train.to_numpy().reshape(-1, 1)
).ravel()

svr_model = SVR(
    kernel="rbf",
    C=100,
    epsilon=0.1
)

svr_model.fit(
    X_train,
    y_train_scaled
)

svr_scaled_pred = svr_model.predict(
    X_test
)

# Convert predictions back to original price scale

svr_pred = target_scaler.inverse_transform(
    svr_scaled_pred.reshape(-1, 1)
).ravel()

svr_mae, svr_mse, svr_rmse, svr_r2 = calculate_metrics(
    y_test,
    svr_pred
)

print("\nModel: Support Vector Regression")
print("Kernel:", "RBF")
print("C      :", 100)
print("MAE     :", svr_mae)
print("MSE     :", svr_mse)
print("RMSE    :", svr_rmse)
print("R²      :", svr_r2)

# =====
# MODEL COMPARISON TABLE
# =====

print("\n" + "=" * 75)
```

```
print("10. REGRESSION MODEL COMPARISON")
print("=" * 75)

results = pd.DataFrame({

    "Model": [
        "Simple Linear Regression",
        "Multiple Linear Regression",
        "Polynomial Regression",
        "Ridge Regression",
        "Lasso Regression",
        "Support Vector Regression"
    ],

    "MAE": [
        simple_mae,
        multiple_mae,
        poly_mae,
        ridge_mae,
        lasso_mae,
        svr_mae
    ],

    "MSE": [
        simple_mse,
        multiple_mse,
        poly_mse,
        ridge_mse,
        lasso_mse,
        svr_mse
    ],

    "RMSE": [
        simple_rmse,
        multiple_rmse,
        poly_rmse,
        ridge_rmse,
        lasso_rmse,
        svr_rmse
    ],

    "R2 Score": [
        simple_r2,
        multiple_r2,
        poly_r2,
        ridge_r2,
        lasso_r2,
        svr_r2
    ]
})

display(
    results.round(4)
)
```

```
# =====
# GRAPH 1 – R2 SCORE COMPARISON
# =====

print("\nR2 Score Comparison:")

plt.figure(
    figsize=(9, 5)
)

bars = plt.bar(
    results["Model"],
    results["R2 Score"]
)

plt.title(
    "R2 Score Comparison of Regression Models",
    fontsize=14,
    pad=12
)

plt.xlabel(
    "Regression Model",
    fontsize=11
)

plt.ylabel(
    "R2 Score",
    fontsize=11
)

plt.xticks(
    rotation=30,
    ha="right",
    fontsize=9
)

for bar in bars:

    height = bar.get_height()

    plt.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height:.3f}",
        ha="center",
        va="bottom",
        fontsize=8
    )

plt.tight_layout()
plt.show()
```

```
# =====  
# GRAPH 2 – RMSE COMPARISON  
# =====  
  
print("\nRMSE Comparison:")  
  
plt.figure(  
    figsize=(9, 5)  
)  
  
bars = plt.bar(  
    results["Model"],  
    results["RMSE"]  
)  
  
plt.title(  
    "RMSE Comparison of Regression Models",  
    fontsize=14,  
    pad=12  
)  
  
plt.xlabel(  
    "Regression Model",  
    fontsize=11  
)  
  
plt.ylabel(  
    "RMSE",  
    fontsize=11  
)  
  
plt.xticks(  
    rotation=30,  
    ha="right",  
    fontsize=9  
)  
  
for bar in bars:  
    height = bar.get_height()  
  
    plt.text(  
        bar.get_x() + bar.get_width() / 2,  
        height,  
        f"{height:,.0f}",  
        ha="center",  
        va="bottom",  
        fontsize=8  
    )  
  
plt.tight_layout()  
plt.show()
```

```
# =====  
# GRAPH 3 – MAE COMPARISON  
# =====  
  
print("\nMAE Comparison:")  
  
plt.figure(  
    figsize=(9, 5)  
)  
  
bars = plt.bar(  
    results["Model"],  
    results["MAE"]  
)  
  
plt.title(  
    "MAE Comparison of Regression Models",  
    fontsize=14,  
    pad=12  
)  
  
plt.xlabel(  
    "Regression Model",  
    fontsize=11  
)  
  
plt.ylabel(  
    "Mean Absolute Error",  
    fontsize=11  
)  
  
plt.xticks(  
    rotation=30,  
    ha="right",  
    fontsize=9  
)  
  
for bar in bars:  
    height = bar.get_height()  
  
    plt.text(  
        bar.get_x() + bar.get_width() / 2,  
        height,  
        f"{height:,.0f}",  
        ha="center",  
        va="bottom",  
        fontsize=8  
    )  
  
plt.tight_layout()  
plt.show()
```

```
# =====  
# GRAPH 4 – ACTUAL VS PREDICTED  
# MULTIPLE LINEAR REGRESSION  
# =====  
  
print("\nActual vs Predicted - Multiple Linear Regression:")  
  
plt.figure(  
    figsize=(7, 6)  
)  
  
plt.scatter(  
    y_test,  
    multiple_pred,  
    alpha=0.65  
)  
  
minimum = min(  
    y_test.min(),  
    multiple_pred.min()  
)  
  
maximum = max(  
    y_test.max(),  
    multiple_pred.max()  
)  
  
plt.plot(  
    [minimum, maximum],  
    [minimum, maximum],  
    "r--",  
    linewidth=2  
)  
  
plt.xlabel(  
    "Actual House Price",  
    fontsize=11  
)  
  
plt.ylabel(  
    "Predicted House Price",  
    fontsize=11  
)  
  
plt.title(  
    "Multiple Linear Regression: Actual vs Predicted",  
    fontsize=13,  
    pad=12  
)  
  
plt.tight_layout()  
plt.show()
```

```
# =====
# GRAPH 5 – ACTUAL VS PREDICTED
# SUPPORT VECTOR REGRESSION
# =====

print("\nActual vs Predicted - Support Vector Regression:")

plt.figure(
    figsize=(7, 6)
)

plt.scatter(
    y_test,
    svr_pred,
    alpha=0.65
)

minimum = min(
    y_test.min(),
    svr_pred.min()
)

maximum = max(
    y_test.max(),
    svr_pred.max()
)

plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    "r--",
    linewidth=2
)

plt.xlabel(
    "Actual House Price",
    fontsize=11
)

plt.ylabel(
    "Predicted House Price",
    fontsize=11
)

plt.title(
    "Support Vector Regression: Actual vs Predicted",
    fontsize=13,
    pad=12
)

plt.tight_layout()
plt.show()
```

```
# =====
# GRAPH 6 – ACTUAL VS PREDICTED
# SIMPLE LINEAR REGRESSION
# =====

print("\nActual vs Predicted - Simple Linear Regression:")

plt.figure(
    figsize=(7, 6)
)

plt.scatter(
    y_test,
    simple_pred,
    alpha=0.65
)

minimum = min(
    y_test.min(),
    simple_pred.min()
)

maximum = max(
    y_test.max(),
    simple_pred.max()
)

plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    "r--",
    linewidth=2
)

plt.xlabel(
    "Actual House Price",
    fontsize=11
)

plt.ylabel(
    "Predicted House Price",
    fontsize=11
)

plt.title(
    "Simple Linear Regression: Actual vs Predicted",
    fontsize=13,
    pad=12
)

plt.tight_layout()
plt.show()
```

```
# =====
# SUMMARY
# =====

print("\n" + "=" * 75)
print("11. FINAL SUMMARY")
print("=" * 75)

print("\nDataset Shape:", house.shape)

print("\nModels Implemented:")
print("1. Simple Linear Regression")
print("2. Multiple Linear Regression")
print("3. Polynomial Regression")
print("4. Ridge Regression")
print("5. Lasso Regression")
print("6. Support Vector Regression")

print("\nEvaluation Metrics:")
print("• Mean Absolute Error (MAE)")
print("• Mean Squared Error (MSE)")
print("• Root Mean Squared Error (RMSE)")
print("• R2 Score")

print("""
Conclusion:
The Housing dataset was prepared for regression analysis by separating
the target variable, encoding categorical features and scaling the input
features. Six regression techniques were implemented and evaluated
using MAE, MSE, RMSE and R2 score. Actual-versus-predicted plots and
model comparison charts were generated to analyze the regression results.
""")

# -----
# SAVE RESULTS
# -----

results.to_csv(
    "housing_regression_results_pragati.csv",
    index=False
)

print("\nModel comparison results saved as:")
print("housing_regression_results_pragati.csv")

print("\n" + "=" * 75)
print("MODULE II COMPLETED SUCCESSFULLY")
print("=" * 75)
```



```
=====
MACHINE LEARNING PRACTICAL - MODULE II
House Price Prediction using Regression Techniques
=====
```

```
Student Name : Pragati Parmar
Roll Number  : BETN1AI25100
Program      : B.Tech CSE (AI & ML)
Subject      : Machine Learning
Dataset      : Housing Dataset
```

```
=====
1. DATASET LOADING AND INSPECTION
=====
```

```
Dataset Shape:
(545, 13)
```

```
First 10 Records:
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
0	13300000	7420	4	2	3	yes	no	no	no	yes	2	yes	furnished
1	12250000	8960	4	4	4	yes	no	no	no	yes	3	no	furnished
2	12250000	9960	3	2	2	yes	no	yes	no	no	2	yes	semi-furnished
3	12215000	7500	4	2	2	yes	no	yes	no	yes	3	yes	furnished
4	11410000	7420	4	1	2	yes	yes	yes	no	yes	2	no	furnished
5	10850000	7500	3	3	1	yes	no	yes	no	yes	2	yes	semi-furnished
6	10150000	8580	4	3	4	yes	no	no	no	yes	2	yes	semi-furnished
7	10150000	16200	5	3	2	yes	no	no	no	no	0	no	unfurnished
8	9870000	8100	4	1	2	yes	yes	yes	no	yes	2	yes	furnished
9	9800000	5750	3	2	4	yes	yes	no	no	yes	1	yes	unfurnished

```
Last 10 Records:
```

	price	area	bedrooms	bathrooms	stories	mainroad	guestroom	basement	hotwaterheating	airconditioning	parking	prefarea	furnishingstatus
535	2100000	3360	2	1	1	yes	no	no	no	no	1	no	unfurnished
536	1960000	3420	5	1	2	no	no	no	no	no	0	no	unfurnished
537	1890000	1700	3	1	2	yes	no	no	no	no	0	no	unfurnished
538	1890000	3649	2	1	1	yes	no	no	no	no	0	no	unfurnished
539	1855000	2990	2	1	1	no	no	no	no	no	1	no	unfurnished
540	1820000	3000	2	1	1	yes	no	yes	no	no	2	no	unfurnished
541	1767150	2400	3	1	1	no	no	no	no	no	0	no	semi-furnished
542	1750000	3620	2	1	1	yes	no	no	no	no	0	no	unfurnished
543	1750000	2910	3	1	1	no	no	no	no	no	0	no	furnished

544 1750000 3850 3 1 2 yes no no no no 0 no unfurnished

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 545 entries, 0 to 544

Data columns (total 13 columns):

#	Column	Non-Null Count	Dtype
0	price	545 non-null	int64
1	area	545 non-null	int64
2	bedrooms	545 non-null	int64
3	bathrooms	545 non-null	int64
4	stories	545 non-null	int64
5	mainroad	545 non-null	object
6	guestroom	545 non-null	object
7	basement	545 non-null	object
8	hotwaterheating	545 non-null	object
9	airconditioning	545 non-null	object
10	parking	545 non-null	int64
11	prefarea	545 non-null	object
12	furnishingstatus	545 non-null	object

dtypes: int64(6), object(7)

memory usage: 55.5+ KB

Descriptive Statistics:

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
price	545.0	NaN	NaN	NaN	4766729.247706	1870439.615657	1750000.0	3430000.0	4340000.0	5740000.0	13300000.0
area	545.0	NaN	NaN	NaN	5150.541284	2170.141023	1650.0	3600.0	4600.0	6360.0	16200.0
bedrooms	545.0	NaN	NaN	NaN	2.965138	0.738064	1.0	2.0	3.0	3.0	6.0
bathrooms	545.0	NaN	NaN	NaN	1.286239	0.50247	1.0	1.0	1.0	2.0	4.0
stories	545.0	NaN	NaN	NaN	1.805505	0.867492	1.0	1.0	2.0	2.0	4.0
mainroad	545	2	yes	468	NaN	NaN	NaN	NaN	NaN	NaN	NaN
guestroom	545	2	no	448	NaN	NaN	NaN	NaN	NaN	NaN	NaN
basement	545	2	no	354	NaN	NaN	NaN	NaN	NaN	NaN	NaN
hotwaterheating	545	2	no	520	NaN	NaN	NaN	NaN	NaN	NaN	NaN
airconditioning	545	2	no	373	NaN	NaN	NaN	NaN	NaN	NaN	NaN
parking	545.0	NaN	NaN	NaN	0.693578	0.861586	0.0	0.0	0.0	1.0	3.0
prefarea	545	2	no	417	NaN	NaN	NaN	NaN	NaN	NaN	NaN
furnishingstatus	545	3	semi-furnished	227	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Missing Values:

	0
price	0
area	0

bedrooms 0
bathrooms 0
stories 0
mainroad 0
guestroom 0
basement 0
hotwaterheating 0
airconditioning 0
parking 0
prefarea 0
furnishingstatus 0

dtype: int64

=====

2. DATA PREPARATION

=====

Target Variable:
price

Original Feature Shape:
(545, 12)

Categorical Columns:
['mainroad', 'guestroom', 'basement', 'hotwaterheating', 'airconditioning', 'prefarea', 'furnishingstatus']

Numerical Columns:
['area', 'bedrooms', 'bathrooms', 'stories', 'parking']

Feature Shape After Encoding:
(545, 13)

Scaled Feature Sample:

	area	bedrooms	bathrooms	stories	parking	mainroad_yes	guestroom_yes	basement_yes	hotwaterheating_yes	airconditioning_yes	prefarea_yes	furnishingstatus_semi-furnished	furr
0	1.046726	1.403419	1.421812	1.378217	1.517692	0.405623	-0.465315	-0.734539	-0.219265	1.472618	1.804941	-0.844888	
1	1.757010	1.403419	5.405809	2.532024	2.679409	0.405623	-0.465315	-0.734539	-0.219265	1.472618	-0.554035	-0.844888	
2	2.218232	0.047278	1.421812	0.224410	1.517692	0.405623	-0.465315	1.361397	-0.219265	-0.679063	1.804941	1.183588	
3	1.083624	1.403419	1.421812	0.224410	2.679409	0.405623	-0.465315	1.361397	-0.219265	1.472618	1.804941	-0.844888	
4	1.046726	1.403419	-0.570187	0.224410	1.517692	0.405623	2.149083	1.361397	-0.219265	1.472618	-0.554035	-0.844888	

=====

3. TRAIN-TEST SPLIT

=====

Training Set:
(381, 13)

Testing Set:
(164, 13)

Training Records: 381
Testing Records : 164

=====

4. SIMPLE LINEAR REGRESSION

=====

Model: Simple Linear Regression
MAE : 1387227.8956867643
MSE : 3193989923757.6206
RMSE : 1787173.725119531
R² : 0.2583140345296001

=====

5. MULTIPLE LINEAR REGRESSION

=====

Model: Multiple Linear Regression
MAE : 920392.9397607954
MSE : 1523019469501.2913
RMSE : 1234106.7496376848
R² : 0.6463350878895877

=====

6. POLYNOMIAL REGRESSION

=====

Model: Polynomial Regression
Degree: 2
MAE : 1001098.2224800427
MSE : 1839776755122.1038
RMSE : 1356383.7049751459
R² : 0.5727799299794265

=====

7. RIDGE REGRESSION

=====

Model: Ridge Regression
Alpha: 1.0
MAE : 920127.6791754975
MSE : 1522791567084.312
RMSE : 1234014.4112141933
R² : 0.6463880097923504

=====

8. LASSO REGRESSION

=====

Model: Lasso Regression
Alpha: 1000
MAE : 920177.6073881944
MSE : 1522791567084.312

RMSE : 1234212.2262627063
R² : 0.6462746312468355

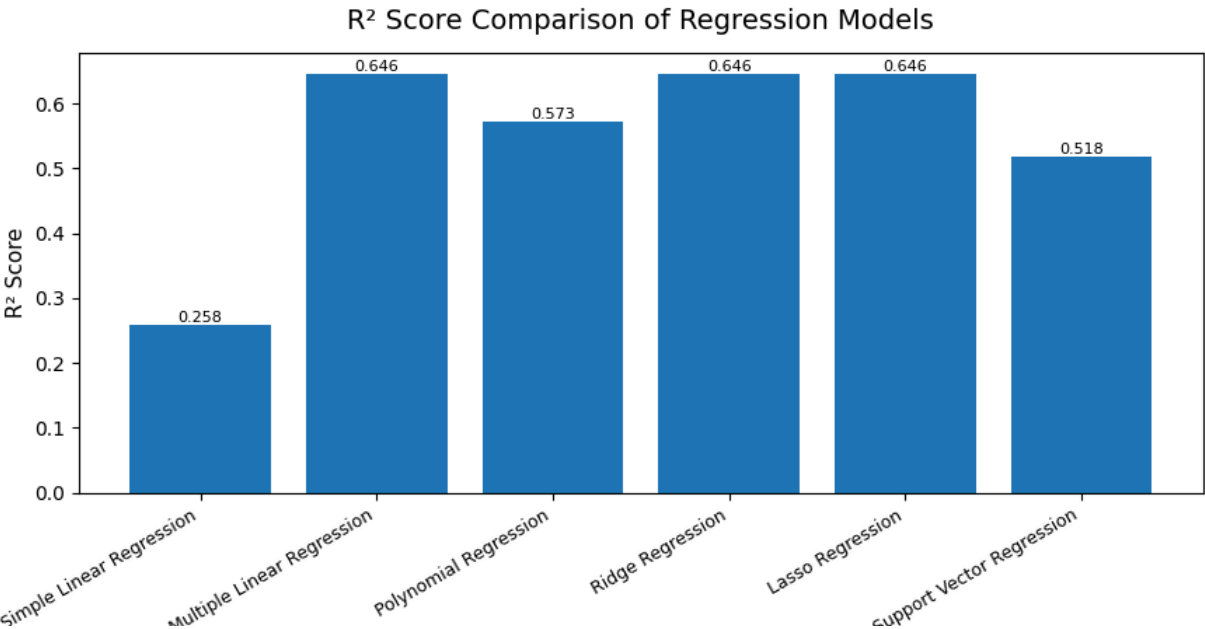
9. SUPPORT VECTOR REGRESSION

Model: Support Vector Regression
Kernel: RBF
C : 100
MAE : 1080637.8181884685
MSE : 2076949742583.894
RMSE : 1441162.635716002
R² : 0.5177052802816756

10. REGRESSION MODEL COMPARISON

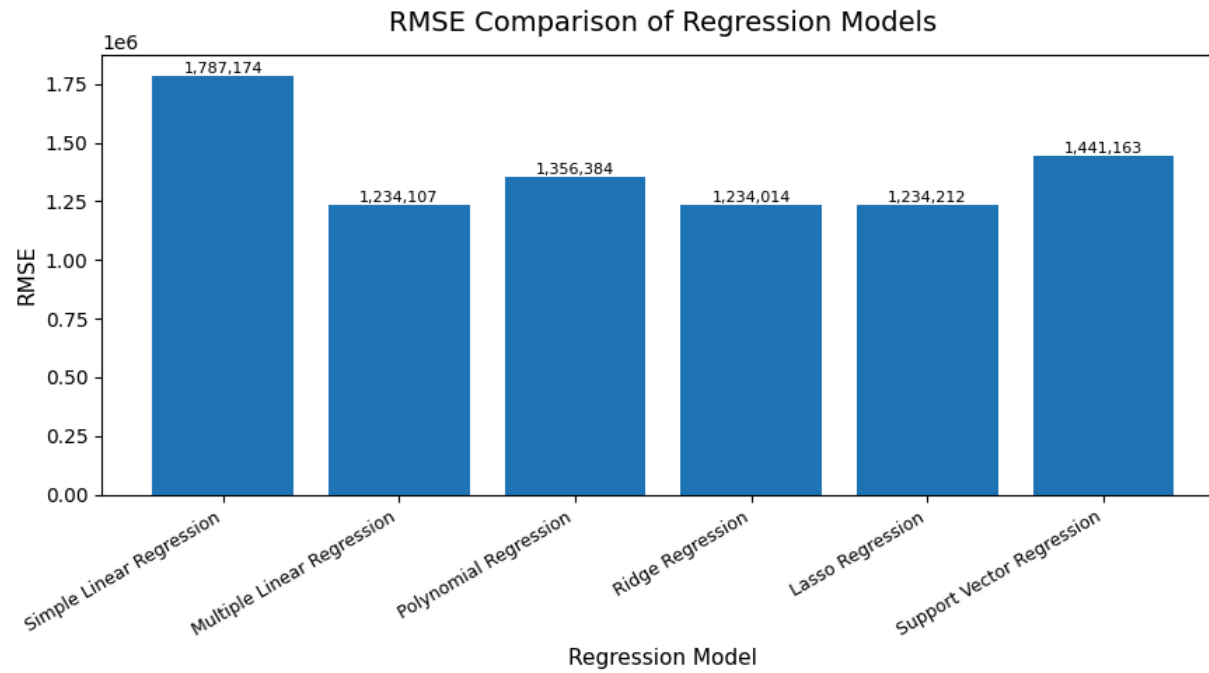
	Model	MAE	MSE	RMSE	R ² Score
0	Simple Linear Regression	1.387228e+06	3.193990e+12	1.787174e+06	0.2583
1	Multiple Linear Regression	9.203929e+05	1.523019e+12	1.234107e+06	0.6463
2	Polynomial Regression	1.001098e+06	1.839777e+12	1.356384e+06	0.5728
3	Ridge Regression	9.201277e+05	1.522792e+12	1.234014e+06	0.6464
4	Lasso Regression	9.201776e+05	1.523280e+12	1.234212e+06	0.6463
5	Support Vector Regression	1.080638e+06	2.076950e+12	1.441163e+06	0.5177

R² Score Comparison:

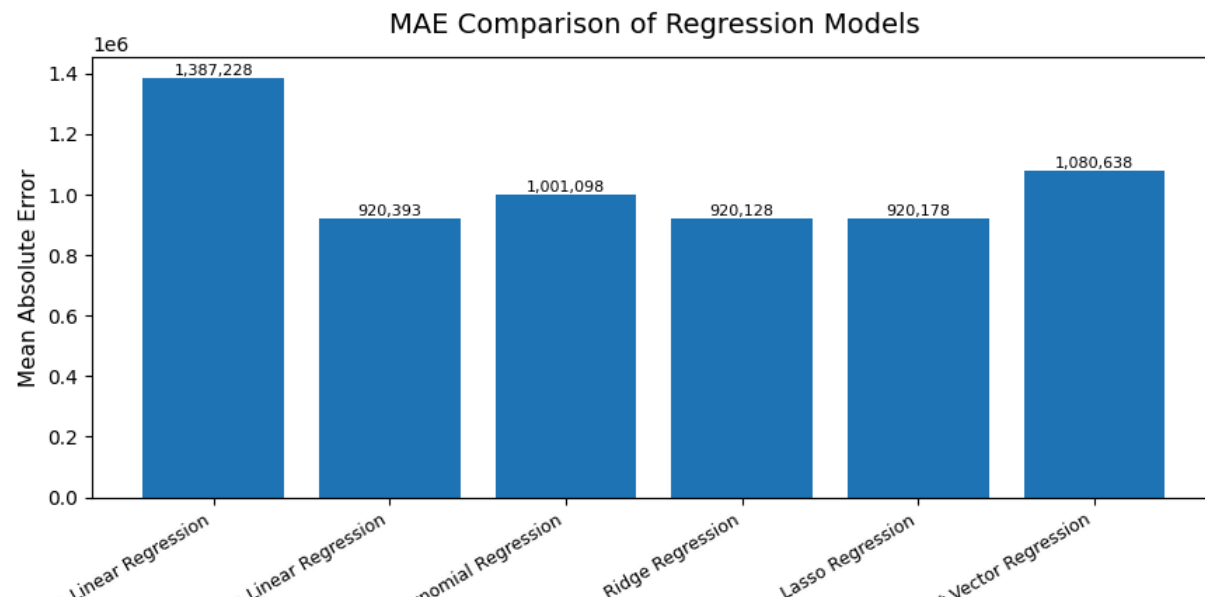


Regression Model

RMSE Comparison:



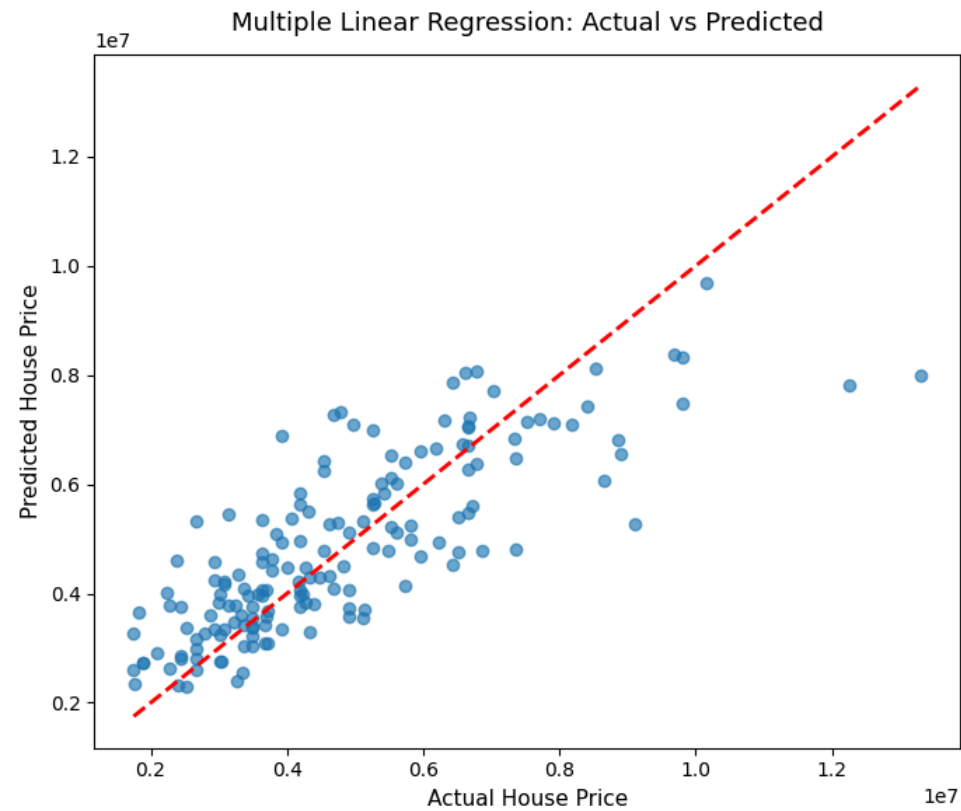
MAE Comparison:



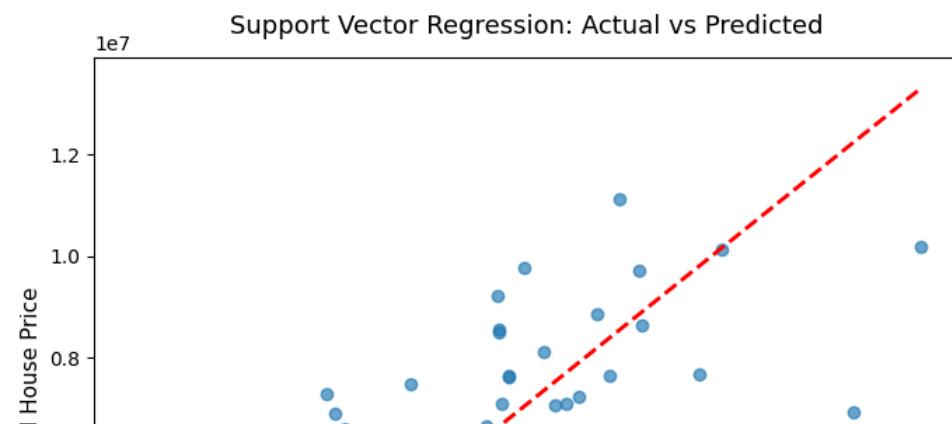
Simple Multiple Poly Support

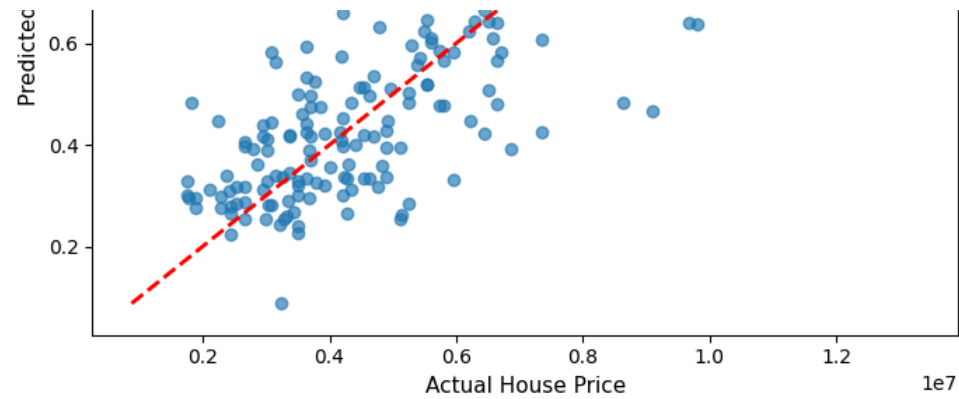
Regression Model

Actual vs Predicted - Multiple Linear Regression:



Actual vs Predicted - Support Vector Regression:





Actual vs Predicted - Simple Linear Regression:

